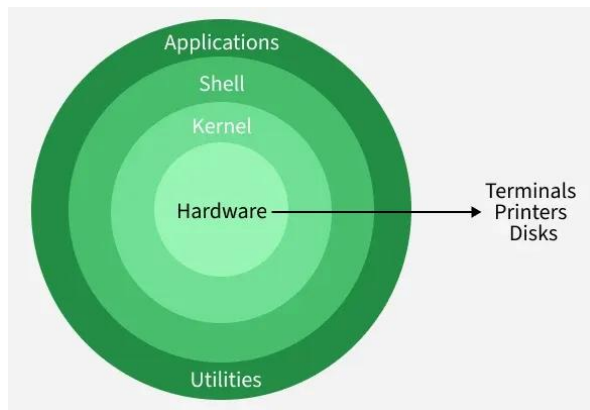


Introduction to Linux

2. Linux is based on the UNIX operating system, a multi-user and multitasking platform developed in the 1970s at AT&T Bell Labs that became the foundation for many modern operating systems. Linux is free and open-source, encouraging global collaboration and innovation while providing strong security, flexibility, and reliable performance across different devices and industries.
 - Provides strong security and stability, making it suitable for servers and development environments.
 - Allows users to freely use, modify, and distribute the operating system.
 - Supports customization for different user requirements and industry applications.
 - Backed by a large global community and a wide range of software tools.

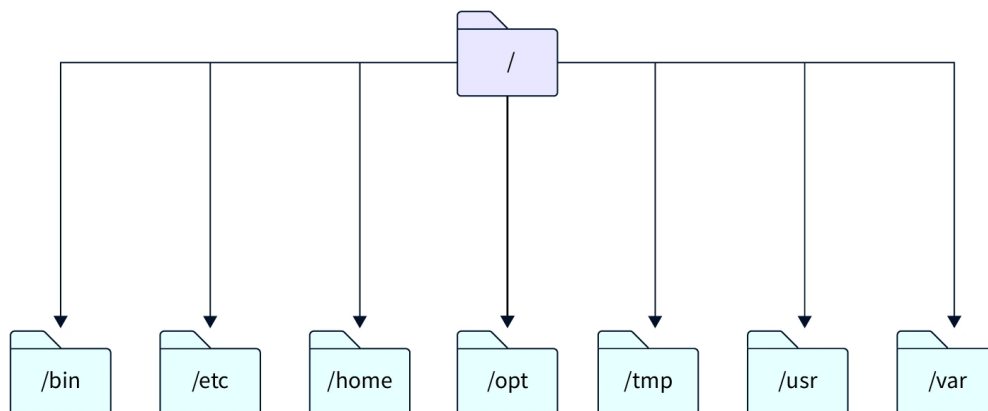


Linux architecture: is the layered structure of the Linux operating system that defines how components like the kernel, shell, system libraries, and hardware interact to manage resources and run programs efficiently. It consists of the following components:



- **Kernel:** Core component of Linux that manages hardware resources, processes, and system operations.
- **System Libraries:** System libraries provide functions that help applications interact with the Linux kernel.
- **Shell:** It is the command-line interface that interprets and executes user commands.
- **Hardware Layer:** Hardware layer includes physical components like CPU, RAM, storage, and input/output devices.
- **System Utilities:** System utilities are built-in tools used for system management, configuration, and maintenance.

Linux File system:



The Root Directory (/)

The root directory is where everything begins in the Linux file system. It's the starting point from which all other directories and subdirectories branch out.

Binary Directory (/bin)

The /bin directory holds essential GNU user-level utilities. These include

1. Core system utilities like 'ls', 'cp', 'rm', 'mv'
2. Command interpreters like 'bash', 'sh'
3. System administration tools like 'sudo', 'su', 'passwd'
4. Text manipulation tools like 'grep', 'sed', 'awk'
5. File compression archiving tools like 'zip', 'gzip'
6. Network utilities like 'ping', 'inconfig', 'ssh'.

Boot Directory (/boot)

The /boot directory contains the Linux kernel, initial RAM disk image (for drivers needed at boot time), and the boot loader. Key files include /boot/grub/grub.conf or menu.lst, which is used to configure the boot loader, and /boot/vmlinuz (or similar), the Linux kernel.

Device Directory (/dev)

The /dev directory is a special one that contains device nodes. Here, the kernel maintains a list of all the devices it understands, adhering to the principle that “Everything is a file”.

System Configuration Directory (/etc)

The /etc directory contains all the system-wide configuration files and a collection of shell scripts that start each of the system services at boot time. Everything in this directory is readable text. While everything in /etc is interesting, here are some all-time favorites: /etc/crontab, a file that defines when automated jobs will run; /etc/fstab, a table of storage devices and their associated mount points; and /etc/passwd, a list of the user accounts

Home Directory (/home)

In normal configurations, each user is given a directory in /home. This limitation protects the system from errant user activity as ordinary users can write files only in their home directories.

Library Directory (/lib)

The /lib directory contains shared library files used by the core system programs. These are similar to dynamic link libraries (DLLs) in Windows.

Lost and Found Directory (/lost+found)

Each formatted partition or device using a Linux file system, such as ext3, will have this directory. It is used in the case of a partial recovery from a file system corruption event. Unless something

really bad has happened to your system, this directory will remain empty.

Media Directory (/media)

On modern Linux systems, the /media directory contains the mount points for removable media such as USB drives, CD-ROMs, and so on, that are mounted automatically at insertion.

Mount Directory (/mnt)

On older Linux systems, the /mnt directory contains mount points for removable devices that have been mounted manually.

Optional Directory (/opt)

The /opt directory is used to install “optional” software, mainly holding commercial software products that might be installed on the system.

Process Directory (/proc)

The /proc directory is special. It’s a virtual file system maintained by the Linux kernel. The “files” it contains are peepholes into the kernel itself. The files are readable and will give you a picture of how the kernel sees your computer.

Root Home Directory (/root)

This is the home directory for the root account.

System Binary Directory (/sbin)

This directory contains “system” binaries, programs that perform vital system tasks that are generally reserved for the superuser.

Temporary Directory (/tmp)

The /tmp directory is intended for the storage of temporary, transient files created by various programs. Some configurations cause this directory to be emptied each time the system is rebooted.

User Directory (/usr)

The /usr directory tree is likely the largest one on a Linux system. It contains all the programs and support files used by regular users.

User Binary Directory (/usr/bin)

/usr/bin contains the executable programs installed by your Linux distribution. It is not uncommon for this directory to hold thousands of programs.

User Library Directory (/usr/lib)

The shared libraries for the programs in /usr/bin are stored in /usr/lib.

Local User Directory (/usr/local)

The /usr/local tree is where programs that are not included with your distribution but are intended for system-wide use are installed. Programs compiled from source code are normally installed in /usr/local/bin. On a newly installed Linux system, this tree exists, but it will be empty until the system administrator puts something in it.

System Administration Directory (/usr/sbin)

This directory contains more system administration programs.

Shared User Directory (/usr/share)

/usr/share contains all the shared data used by programs in /usr/bin. This includes things such as default configuration files, icons, screen backgrounds, sound files, and so on.

Documentation Directory (/usr/share/doc)

Most packages installed on the system will include some kind of documentation. In /usr/share/doc, we will find documentation files organized by package.

Variable Directory (/var)

With the exception of /tmp and /home, the directories we have looked at so far remain relatively static; that is, their contents don't change. The /var directory tree is where data that is likely to change is stored. Various databases, spool files, user mail, and so forth, are located here.

Log Directory (/var/log)

/var/log contains log files, records of various system activity. These are important and should be monitored from time to time. The most useful ones are /var/log/messages and /var/log/syslog. Note that for security reasons on some systems, you must be the superuser to view log files.

Basic Terminal Navigation in Linux:

1. What is the Terminal?

The terminal (or command line interface) is our direct communication channel with the Linux operating system. Instead of clicking through folders using a GUI, we can type commands to interact with files, directories, and processes. For developers and system administrators, the terminal is an essential tool — powerful, fast, and scriptable.

2. Navigating the File System

`pwd` - **Print Working Directory**

Displays the current directory path where we are working.

```
pwd
```

Example output:

```
/home/jeetesh
```

This tells us that our terminal is currently inside the `/home/jeetesh` directory.

`ls` - List Files and Directories

Lists all files and folders in our current directory.

```
ls
```

Useful options:

Press enter or click to view image in full size

Option	Description
<code>ls -l</code>	Shows detailed info - permissions, size, date, owner
<code>ls -a</code>	Displays hidden files (starting with <code>.</code>)
<code>ls -lh</code>	Shows human-readable file sizes (KB, MB)

Tip: When working on projects or servers, `ls -l` is especially useful for checking file permissions and ownership.

`cd` - Change Directory

Used to move between directories.

```
cd /home/nakul/Documents
```

Other variations:

```
cd ..  # Go one directory up  
cd     # Go back to the home directory
```

Pro Tip: Use the Tab key for auto-completion of directory names — it saves a lot of time!

3. Viewing File Content

`cat` - View Entire File

Used to display the content of a file.

```
cat notes.txt
```

It will print the full content directly in the terminal.

Note: For very large files, `cat` may cause content to scroll off the screen - that's when we use `less`.

`less` - Scroll Through File Content

Used to view large files one page at a time.

```
less filename.txt
```

Navigation inside less:

Key	Action
↑ / ↓	Scroll line by line
b	Scroll one page up
Spacebar	Scroll one page down
g	Go to the first line of file
G	Go to the last line of file
Spacebar	Scroll one page down
q	Quit and return to terminal

Example: View system logs with:

```
less /var/log/syslog
```

4. Cleaning the Terminal

`clear` - Clear the Screen

Cleans the terminal window for a fresh start.

```
clear
```

Shortcut: `Ctrl + L` Both commands achieve the same result.

5. Checking Command History

`history` - Show Previous Commands

Displays all the commands we've recently used.

```
history
```

We can rerun commands from history:

```
!!    # Repeats the last command
!45   # Executes command number 45 from the history list
```

This is very helpful when repeating long commands during development or debugging.

6. Combining Commands Using Pipes

Linux allows us to chain commands using the pipe operator (`|`). The output of one command becomes the input of another.

Example:

```
ls -l | less
```

This command lists files in long format (`ls -l`) and pipes the output to `less`, so we can scroll through it easily.

File And Directory Management Using Linux Commands

Creating Files

- The `touch` command is used to create a new file in Linux.
- If the specified file does not exist, touch creates a new blank file.
- If the file already exists, its contents remain unaffected, and only the file's timestamp may be updated.

Example:

```
touch filename
```

Displaying File Contents

- The `cat` command is used to display the contents of a file in the terminal.
- For large files, the output may scroll past the screen too quickly; in such cases, commands like `more` or `less` can be used to view the content page by page.

Example:

```
cat filename
```

4. Copying a File

- The `cp` command is used to create a copy of a file in Linux.
- It copies the contents of the source file to a new file at the specified destination.
- The new file will have the same name and content as the original, unless a different name is specified.

Syntax:

```
cp source/filename destination/
```

Example:

```
cp source/filename destination/
```

5. Moving a File

- The **mv** command is used to move a file from one location to another in Linux.
- It moves the file from the source directory and places it in the destination directory with the same name and content.

Example:

```
mv source/filename destination/
```

6. Renaming a File

- The **mv** command can also be used to rename a file in Linux. It changes the file name from filename to new_filename while retaining the file's content. Essentially, the original file is replaced with a file of the new name without altering the data.

```
mv filename new_filename
```

7. Deleting a File

The **rm** command is used to delete a file in Linux. It permanently removes the specified file from the directory. Use this command carefully, as deleted files cannot be easily recovered.

Example:

```
rm filename
```

File Permission and Ownership:

File ownership and permissions in Linux define who owns a file and who is allowed to read, write or execute it. Every file and directory is associated with a user owner and a group owner and set of permissions apply to the owner, group members and all other users. This answers the question 'who is allowed to do what on a resource'.

Directories can be likened to containers and indexing structure for files and other directories for locating where files are stored.

Permissions Types

Linux file permissions types define the operations that can be performed on files and directories. The permission model consists fundamentally of three types: read (r), write (w) and execute (x) in combination control data access.

Permissions	Effect on Files	Effect on Directories
Read (r)	Allows viewing file contents	Allows listing directory contents
Write (w)	Allows modifying file contents	Allows creating, deleting and renaming files within the directory
Execute (x)	Allows executing the file as a program or script	Allows entering the directory and accessing files

Permission ownership

This implies to the user or group assigned to a file or directory. Each filesystem object (files and directories) is owned by a specific user

and group. Ownership defines access or denial of access to user object find files and directories. Every file and directory has permission types and ownership by default which can viewed using `ls -l`.

Permission ownership is in 3 sections:

owner (u) : the user account that creates the file or directory

group (g): the group the specific user belongs to

other (o): account not in the same group as the user that the owns a file

Changing Permissions:

chmod command is used in modifying file and directory permissions.

Types of chmod settings

Symbolic mode uses permission components and operators to assign privileges.

User classes	Operation	Permissions
owner (u), group (g), other (o)	add (+), remove (-)	read (r), write (w), execute (x)

chmod u+rwx, g+rw, o+r adds read, write and execute to the owner, read and write to the group and read only to other user account not included in the group

chmod g+w, o-x adds write permission to the group assigned with the file but revokes the execute permission to other user accounts that are not the owner or in the group assigned to the file.

2. Numerical mode uses numbers to represent permissions defined by digits. The numerals are considered to be in the octal form

Value	Meaning
Read	4
Write	2
Execute	1

chmod 754 adds r+w+x for owner, r+x for group, r for other user account. chmod 524 adds r+x for owner, w for group, r for owner. chmod 642 adds r+w for owner r for group, w for other.

Octal	Symbolic	Meaning
0	---	No permissions given
1	--x	Execute only
2	-w-	Write only
3	-wx	Write + Execute
4	r—	Read only
5	r-x	Read + Execute
6	rw-	Read + Write
7	rwX	Read + Write + Execute

A simple comparison between the symbolic and numeric mode of file permission

Linux Process Management:

The ps command comes to your aid in the realm of Linux when you want to discover what applications are running on your computer and acquire information about them. Ps stands for

“Process Status,” and based on the parameters you select, it allows you to view information on running processes, including their distinct IDs (PIDs) and other data. It reads information about processes from virtual files in the virtual file system known as /proc. Ps gives you insights into the processes that are now executing on your system and lets you tailor its output to your needs using a variety of choices.

ps

Linux gives us a tool called ps — an acronym for “Process Status” — for viewing data pertaining to the processes on a system. The ps command is used to list the active processes, their PIDs, and additional data depending on the arguments selected. From the virtual files in the /proc file system, it reads the process information. Because /proc contains virtual files, it is known as a virtual file system. Ps offers a wide range of options for modifying the output to suit our needs.

Syntax

```
ps [options]
```

- 3.PID:** It’s like a special number assigned to each running program. It helps us identify which program is which. Think of it as a unique ID for each program.
- 4.TTY:** It tells us which “room” or terminal a program is running in. A terminal is where you type commands and

see the output. So, TTY shows the room where the program is doing its work.

5. **TIME:** It shows how much “time” a program has been using the computer’s brain (CPU) to do its work. The time is displayed in minutes and seconds. If it shows 00:00:00, it means the program hasn’t used the computer’s brain yet.

6. **CMD:** It’s like the name of the command that made the program start working. For example, if a program was started using the command “ls” to list files, you will see “ls” in the CMD column.

Options

`-a`: Shows processes for all users, including those without a terminal (TTY).

- Example: `ps -a`

```
noransaber2023@Ahmed:~/alx-system_engineering-devops/0x05-processes_and_signals$ ps -a
  PID TTY          TIME CMD
  475 pts/1    00:00:00 bash
 1609 pts/0    00:00:00 ps
noransaber2023@Ahmed:~/alx-system_engineering-devops/0x05-processes_and_signals$ |
```

`-e`: Displays all processes, same as `-A`.

- Example: `ps -e`

```

noransaber2023@Ahmed:~/alx-system_engineering-devops/0x05-processes_and_signals$ ps -e
  PID TTY          TIME CMD
    1 ?           00:00:03 systemd
    2 ?           00:00:00 init-systemd(Ub
    5 ?           00:00:00 init
   41 ?           00:00:08 systemd-journal
   68 ?           00:00:02 systemd-udevd
   90 ?           00:00:00 snapfuse
   91 ?           00:00:01 snapfuse
   94 ?           00:00:00 snapfuse
   95 ?           00:00:00 snapfuse
   98 ?           00:00:05 snapfuse
   99 ?           00:00:00 snapfuse
  103 ?           00:00:00 snapfuse
  104 ?           00:00:01 snapfuse
  111 ?           00:00:00 systemd-resolve
  147 ?           00:00:00 cron
  150 ?           00:00:00 dbus-daemon
  190 ?           00:00:00 networkd-dispat
  191 ?           00:00:00 rsyslogd
  192 ?           00:00:03 snapd
  195 ?           00:00:00 systemd-logind
  218 ?           00:00:00 subiquity-serve
  224 hvc0          00:00:00 agetty
  231 tty1         00:00:00 agetty
  232 ?           00:00:00 unattended-upgr
  404 ?           00:00:00 SessionLeader
  405 ?           00:00:00 Relay(408)
  408 pts/0        00:00:00 bash
  409 pts/1        00:00:00 login
  430 ?           00:00:16 python3.10
  469 ?           00:00:00 systemd
  470 ?           00:00:00 (sd-pam)
  475 pts/1        00:00:00 bash
  490 ?           00:00:20 python3
 1623 pts/0        00:00:00 ps
noransaber2023@Ahmed:~/alx-system_engineering-devops/0x05-processes_and_signals$ |

```

`-f`: Provides a full-format listing of processes with more details.

- Example: `ps -f`

```

noransaber2023@Ahmed:~/alx-system_engineering-devops/0x05-processes_and_signals$ ps -f
UID          PID    PPID  C STIME TTY          TIME CMD
noransa+    408      405  0 10:31 pts/0      00:00:00 -bash
noransa+   1626      408  0 15:07 pts/0      00:00:00 ps -f

```

`-l`: Displays long-format output with additional process information.

- Example: `ps -l`

```
noransaber2023@Ahmed:~/alx-system_engineering-devops/0x05-processes_and_signals$ ps -l
F S  UID      PID     PPID  C  PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S  1000      408       405  0   80   0  -  1600 do_wai pts/0        00:00:00 bash
0 R  1000     1627       408  0   80   0  -  1870 -      pts/0        00:00:00 ps
```

`-x`: Lists all processes, including those without a terminal (TTY).

- Example: `ps -x`

```
noransaber2023@Ahmed:~/alx-system_engineering-devops/0x05-processes_and_signals$ ps -x
  PID TTY          STAT       TIME COMMAND
  408 pts/0      Ss          0:00 -bash
  469 ?          Ss          0:00 /lib/systemd/systemd --user
  470 ?          S           0:00 (sd-pam)
  475 pts/1      S+          0:00 -bash
 1632 pts/0      R+          0:00 ps -x
noransaber2023@Ahmed:~/alx-system_engineering-devops/0x05-processes_and_signals$ |
```

`-p`: Displays information for specific process IDs (PIDs).

- Example: `ps -p PID`

`-r`: Sorts processes in reverse order by their CPU usage.

- Example: `ps -r`

```
noransaber2023@Ahmed:~/alx-system_engineering-devops/0x05-processes_and_signals$ ps -r
  PID TTY          STAT       TIME COMMAND
 1633 pts/0      R+          0:00 ps -r
noransaber2023@Ahmed:~/alx-system_engineering-devops/0x05-processes_and_signals$ |
```

`-o`: Customizes the output format, letting you choose specific columns.

- Example: `ps -o pid,ppid,cmd`

-H: Shows a process hierarchy by indenting child processes under their parent process.

-T: View process associated with terminal

```
noransaber2023@Ahmed:~/alx-system_engineering-devops/0x05-processes_and_signals$ ps -T
  PID   SPID TTY          TIME CMD
   408    408 pts/0    00:00:00 bash
  1652   1652 pts/0    00:00:00 ps
noransaber2023@Ahmed:~/alx-system_engineering-devops/0x05-processes_and_signals$ |
```

Stop a process

Kill

Use the 'kill' command in Linux to terminate a process. The process receives a signal from the kill command.

There are numerous signal types that you can use. The most typical one is "kill -9," which is "SIGKILL."

All the signals can be listed using:

```
kill -l
```

```
noransaber2023@Ahmed:~/alx-system_engineering-devops/0x05-processes_and_signals$ kill -L
 1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
 6) SIGABRT     7) SIGBUS     8) SIGFPE     9) SIGKILL    10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM    15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD   18) SIGCONT    19) SIGSTOP    20) SIGTSTP
21) SIGTTIN    22) SIGTTOU    23) SIGURG     24) SIGXCPU    25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF   28) SIGWINCH   29) SIGIO      30) SIGPWR
31) SIGSYS     34) SIGRTMIN   35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX
```

SIGTERM, the default signal at 15, is used. It follows that using the kill command alone, without specifying a number, causes the SIGTERM signal to be sent.

The syntax for killing a process is:

```
kill [pid]
```

or

```
kill -9 [pid]
```

A ‘SIGKILL’ signal will be sent to the process using this command. In the event that the process rejects a standard kill request, this should be utilised.

Three methods can be used to specify signals:

There are three ways to specify signals, and they are as follows:

1) By number: A signal can be described by a number. As an illustration, let’s say we have the PID “1212” and want to kill it with a “SIGKILL” signal. ‘9’ is the signal number for SIGKILL (use the ‘kill -l’ command to obtain signal numbers).

```
kill -9 1212
```

2) We can also define signal using SIG prefix (for example, -SIGkill).

For instance, the signal to send is SIGTERM, and the PID is 1432. Use the 'kill -l' command to only examine the signal number of the 'SIGTERM' signal.

```
kill -SIGTERM 1432
```

Without the use of the SIG prefix, we can also specify signals.

For instance, if you want to send the signals "TERM" and "1234". We can use the 'kill -l' command to simply check the signal number of the 'TERM' signal.

```
kill -TERM 1234
```

The 'kill' command in Linux: Some Common Signals

Signal Name	Signal Number	Description
SIGHUP	1	It hangup detected on controlling terminals or death of controlling process.
SIGINT	2	It interrupts from keyboard.
SIGKILL	9	It kills signal.
SIGTERM	15	It terminates signal.

kill PID

This option specifies the process ID of the process to be killed.

```
kill pid
```

kill -s

The signal that will be transmitted to the process is specified by this option.

```
kill {-signal | -s signal} pid
```

pkill

A Unix/Linux tool called pkill can deliver signals to processes based on their names or patterns. With just one command, you can signal or stop many processes that match a certain name or set of criteria.

Syntax

```
pgkill [options] pattern
```

- Here, “pattern” refers to the name or pattern of the processes you want to target. `pgkill` will search for all processes whose names match the specified pattern and send them a signal.
- We used an extended regular expression to specify the matched PATTERN>.
- The `pgkill` command is run with the `-signal` argument if a process matches an alternative signal. An alternative method of transmitting signals is to run `pgkill` followed by prefixing the number or signal name (-)

Press enter or click to view image in full size

15 (TERM):	For stopping any process completely
9 (KILL):	For killing a process
1 (HUP):	For reloading a process

- When you want to send a signal to a process (like asking it to terminate), you have three ways to specify the signal:

7. Without “SIG” prefix: You can use the signal name without the “SIG” prefix. For example, to send the “hang-

up” signal, you can use -HUP. It’s a shorter way to represent the signal.

8. With “SIG” prefix: You can also use the signal name with the “SIG” prefix. For example, to send the “hang-up” signal, you can use -SIGHUP. This way is more explicit and follows a standard naming convention.

9. Using a number: Each signal has a unique number associated with it. Instead of using the signal name, you can directly use its number. For example, -1 represents the “hang-up” signal.

Killing the process by its name

Pkill begins terminating all running programs by delivering the 15 (TERM) signal when it is called without any arguments. Run the following command to gracefully terminate all Firefox processes:

```
pkill -15 firefox
```

- When at least one active process with the given name exists, the command returns 0. Otherwise, exit code 1 is required. When writing shell scripts, this can be helpful.

Kill the Process Begins with a Specific Expression and Ends Therewith To match process names

use the pkill regular expression. It is more advantageous to use the pgrep command if you print the matched processes first before sending one. For instance, the following command will display anything that begins with “ssh”:

```
pkill '^ssh$'
```

- **Note:** The caret (^) character matches at the beginning of the string, and the dollar \$ at the end.

Pkill by default just checks against the process name. The command matches against entire argument lists when the -f option is provided. Quote the full command if it contains spaces:

```
pkill -9 -f "ping 8.8.8.8"
```

To instruct pkill to match processes being run by a certain user, use the -u option:

```
pkill -u noran
```

Multiple users can be specified by separating their names with commas:

```
pkill -u noran,abdo
```

- Additionally, search patterns and choices can be combined. For instance, you might input the following to send the KILL signal to all processes with the names “gnome” and running as the user “mark”:

```
pkill -9 -u noram gnome
```

- Use the -n (for newest) or -o (for oldest) option to display only the most recent (newest) or least recent (oldest) started processes, respectively.

```
pkill -9 -n screen
```

Searching

pgrep

pgrep: Imagine you have a list of people (processes) doing different jobs (tasks) in your house (computer). If you want to find out who is doing a specific job, like cleaning the room, you can use `pgrep`. It looks through the list of all people (processes) and tells you the ID of the person (process ID) who is doing that particular job (matching the name you give).

`pgrep` is a command-line tool used to find and list processes on a computer based on certain criteria, such as process name, user ID, terminal, and more.

Syntax

```
pgrep [options] pattern
```

Options

`-l, --list-name`: Lists the Process ID (PID) along with the process name.

Example 1:

```
pgrep -l firefox
```

- will list all processes with "firefox" in their names along with their PIDs.

```
1234 firefox
5678 firefox
```

`-a, --list-full`: Lists the PID and the full command line of the processes.

Example 2:

```
# Example 2: List the full command line along with the PID
pgrep -a python
```

- will list all processes with "python" in their names along with their PIDs and full command lines.

Press enter or click to view image in full size

```
9876 python script1.py
5432 /usr/bin/python3 script2.py
```

`-v, --inverse`: Negates the matching. It will list processes that do not match the given pattern.

Example 3:

```
pgrep -v chrome
```

- will list all processes that do not have "chrome" in their names.

```
noransaber2023@Ahmed:~$ pgrep -v chrome
1
2
5
41
68
82
89
90
91
92
93
97
101
123
145
146
149
151
158
188
214
226
232
234
380
381
384
385
443
444
450
452
473
noransaber2023@Ahmed:~$
```

`-c, --count`: Gives the count of matching processes.

Example 4:

```
pgrep -c notepad
```

- will display the number of processes with "notepad" in their names.

```
noransaber2023@Ahmed:~$ pgrep -c notepad
0
noransaber2023@Ahmed:~$ |
```

`-u, --uid <ID, ...>`: Matches processes by their real user IDs.

Example 5:

```
pgrep -u 1000
```

- will list processes owned by the user with ID 1000.

```
noransaber2023@Ahmed:~$ pgrep -u 1000
384
443
444
452
noransaber2023@Ahmed:~$ |
```

`-t, --terminal <tty, ...>`: Matches processes based on the controlling terminal.

Example 6:

```
pgrep -t pts/0
```

- will list processes running on terminal "pts/o".

```
noransaber2023@Ahmed:~$ pgrep -t pts/0
384
noransaber2023@Ahmed:~$
```


`-x, --exact`: Matches exactly with the command name (no partial matches).

Example 7:

```
pgrep -x bash
```

- will list processes with the exact name "bash" and not "bashrc" or "bash_profile."